Peripheral, Driver & Device-Control Management for Enterprise Desktops (Intune)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary 4

The problem . . . 4

The solution . . . 4

What's included . . . 4

The outcome . . . 4

Solution Overview 5

What this solution does . . . 5

How Intune manages each area . . . 5

The standard deployment model . . . 6

Object naming convention (used verbatim throughout) . . . 6

What you get . . . 7

Requirements 7

R1. Windows devices enrolled and reporting update status . . . 7

R2. Windows edition and licensing for driver update policies . . . 8

R3. Driver source strategy (Windows Update vs. vendor packages) . . . 8

R4. Printer delivery method (Universal Print or queue package) . . . 8

R5. Microsoft Defender device-control prerequisites . . . 9

R6. Removable-storage baseline inventory . . . 9

R7. Pilot and production security groups . . . 9

R8. A change/rollback window and communications . . . 10

Global Requirements — Intune Solutions (category-level) 10

G1. Microsoft Entra ID tenant . . . 10

G2. Microsoft Intune subscription . . . 10

G3. Administrative roles . . . 10

G4. Device enrollment in Intune . . . 10

G5. Pilot security group . . . 11

G6. Supported Windows baseline . . . 11

G7. Network access to Microsoft cloud endpoints . . . 11

Implementation Guide 11

Stage 0 — Assess (read-only) . . . 11

Stage 1 — Create the groups . . . 12

Stage 2 — Configure Windows driver update management . . . 12

Stage 3 — Configure peripheral device-installation behavior . . . 13

Stage 4 — Deploy printer / peripheral configuration . . . 13

Stage 5 — Configure device control (USB / removable storage) . . . 14

Stage 6 — Validate, then expand to production . . . 15
Testing — Pass/Fail Gates 15
Gate 1 — Driver current . . . 15
Gate 2 — Printer mapped . . . 16
Gate 3 — USB / removable-storage policy enforced . . . 16
Coverage check (before expanding) . . . 17
Post-expansion check (production) . . . 17
Troubleshooting 17
Drivers . . . 17
Peripherals & printers . . . 18
Device control (USB / removable) . . . 18
Coverage, assignment, and scripts . . . 19
Best Practices 20
Staged driver rollout . . . 20
Device control (USB / removable) exceptions . . . 20
Peripherals and printers . . . 21
Operating the solution . . . 21
Deliverables Summary 21
Documents . . . 21
PowerShell scripts . . . 22
Object naming convention (reused everywhere) . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary

The problem

Enterprise desktops carry three recurring, expensive risks that are hard to control at scale:

- **Drivers** — an unmanaged driver update can break a whole fleet, and Windows Update left to itself gives no review or staging.
- **Peripherals and printers** — printers that won't map and unpredictable peripheral installs generate a constant stream of help-desk tickets.
- **Removable/USB storage** — uncontrolled USB is a data-exfiltration and malware vector, and a direct compliance finding in regulated environments.

Solved reactively, each is a recurring cost. Solved once, with a repeatable method, they become a managed, auditable capability.

The solution

A complete, self-guided package to **manage drivers, peripherals/printers, and removable-device control from Microsoft Intune** — the same repeatable method a consultant would use, productized so your team can execute it directly:

- **Windows driver update management** to review, approve, and stage drivers safely instead of letting devices pull whatever is offered.
- **Configuration profiles and printer delivery** (Universal Print or queue package) to make the right peripherals and printers available predictably.
- **Microsoft Defender device control / removable-storage policy** to allow, audit, or block USB and removable storage by policy, with a defined exception path for approved drives.

Everything is deployed **pilot-first** using a **named** pilot group, validated against explicit **pass/fail gates**, and only then expanded to production — and every change is **rollback-ready**.

What's included

- Seven guides: Solution Overview, Requirements (verify + implement each prerequisite), Implementation, Testing, Troubleshooting, Best Practices, and a Deliverables Summary.
- Five PowerShell scripts (Microsoft Graph): discovery, per-device validation, coverage reporting, safe remediation (defaults to preview mode), and pilot-group targeting.
- A consistent object naming convention reused across every guide and script.

The outcome

Drivers current, peripherals working, and removable devices controlled across the fleet — managed from Intune, piloted before production, validated, and reversible at every step. Self-guided and tested, at a fraction of the cost of an equivalent consulting engagement, using a method and scripts you keep and reuse.

Solution Overview

What this solution does

Enterprise desktops depend on three things that are easy to break and hard to control at scale: **drivers**, **peripherals and printers**, and **removable/USB device access**. A single bad driver can knock over a fleet; a mismapped printer generates a steady stream of help-desk tickets; and an uncontrolled USB port is a data-exfiltration and malware risk that regulators care about.

This solution gives you a **repeatable, pilot-first way to manage all three from Microsoft Intune**:

1. **Drivers** — using **Windows driver update management** in Intune (the Windows Autopatch / Windows Update for Business **driver update policy**) to review, approve, and stage driver and firmware updates safely rather than letting every device pull whatever Windows Update offers.

2. **Peripherals and printers** — using Intune **configuration profiles** (Settings Catalog and templates) to control how devices connect to printers and peripherals, plus a Win32/Universal-Print-friendly approach to mapping printers where needed.

3. **Device control (removable/USB access)** — using **Microsoft Defender device control** and the **Administrative Templates / Settings Catalog removable-storage** settings, deployed through Intune, to allow, audit, or block removable storage and USB devices by policy.

Everything is deployed **to a named pilot group first**, validated against **pass/fail gates**, and only then expanded to production — so a mistake never reaches the whole fleet at once, and every change is rollback-ready.

How Intune manages each area

Drivers — Windows driver update management

Windows Update for Business (surfaced in Intune as **Driver update policies** under **Devices** → **Windows** → **Manage updates** → **Windows Driver updates**) lets you choose between two approval modes:

- **Automatically approve all** driver updates (with an optional deferral), or
- **Manually approve and deploy** specific driver updates after review.

For enterprise desktops the safe pattern is **manual approval for the pilot ring**, then automatic (or reviewed) approval for production once a driver has soaked. Intune shows you which drivers are **recommended** vs **other**, how many devices need each, and lets you approve a driver with a scheduled make-available date. This replaces "hope Windows Update does the right thing" with an auditable, staged process.

Peripherals and printers

Peripheral behavior is controlled with **configuration profiles**:

- A **Settings Catalog** profile controls connection and device-installation behavior (for example, allowing or restricting installation of devices by setup class, and controlling USB behavior that affects peripherals).

- **Printer mapping** is delivered either through **Universal Print** (cloud printing, no print server) or, where a traditional queue is required, a small deployment package. This solution documents the Intune-side assignment and the settings that keep peripheral installation predictable.

Device control — removable storage / USB access

Two complementary Intune surfaces control removable and USB devices:

- **Defender device control** (endpoint security / Settings Catalog **Defender** → **Device Control** settings) provides rich **allow / audit / deny** rules for removable storage, printers, and other device families, with **read / write / execute** granularity and per-group exceptions.
- The **Administrative Templates / Settings Catalog "Removable Storage Access"** settings provide a simpler blanket **deny write / deny read / deny execute** control when you don't need per-device rules.

You choose the surface that matches your requirement: a blanket "block USB write" is a template setting; "block all removable storage except approved encrypted drives for the Finance group" is a Defender device-control policy with an exception group.

The standard deployment model

Every part of this solution follows the same lifecycle, with a **pass/fail gate** at each stage:

Assess → **Configure** → **Pilot** → **Validate** → **Expand**

- **Assess** — inventory current driver state, printers/peripherals in use, and existing removable-storage exposure (the Discovery script does this read-only).
- **Configure** — create the named policies, profiles, and groups in Intune (nothing assigned to production yet).
- **Pilot** — assign everything to the **pilot group** (`SG-PERIDEV-Pilot`) only.
- **Validate** — confirm the pass/fail gates on pilot devices: drivers current, printer mapped, USB policy enforced.
- **Expand** — assign to the **production group** (`SG-PERIDEV-Production`) once the gates pass; keep the pilot ahead of production for future changes.

Object naming convention (used verbatim throughout)

To keep the tenant clean and the scripts predictable, this solution uses one naming convention and **reuses these exact names** in every guide and script:

Object	Name
Pilot security group	`SG-PERIDEV-Pilot`
Production security group	`SG-PERIDEV-Production`
USB exception (allowed-drives) group	`SG-PERIDEV-USB-Exception`
Driver update policy — pilot	`PERIDEV-DriverUpdates-Pilot`
Driver update policy — production	`PERIDEV-DriverUpdates-Prod`

Object	Name
Printer/peripheral config profile — pilot	`PERIDEV-Printers-Pilot`
Peripheral device-installation profile	`PERIDEV-Peripherals-DeviceInstall`
Device-control (USB) policy — pilot	`PERIDEV-DeviceControl-USB-Pilot`
Device-control (USB) policy — production	`PERIDEV-DeviceControl-USB-Prod`

Wherever a later document says "assign `PERIDEV-DeviceControl-USB-Pilot`," it means exactly that object — do not rename it.

What you get

- Click-by-click Intune configuration for drivers, peripherals/printers, and device control.
- A requirements guide that tells you how to **verify** and **implement** each prerequisite.
- Five PowerShell scripts (Graph) for discovery, per-device validation, coverage reporting, safe remediation, and pilot-group targeting.
- Testing pass/fail gates and a troubleshooting guide.
- Best practices for staged driver rollout and handling exceptions.

Requirements

Before you configure anything, confirm the prerequisites below. The **category-level global requirements** (Entra tenant, Intune subscription, admin roles, enrollment, supported Windows baseline, network access) are documented separately in *Global Requirements — Intune Solutions* and are assumed here. This page covers the prerequisites **specific to peripheral, driver, and device-control management**.

Each requirement is written as **Description / Why / Verify / Implement if missing.**

R1. Windows devices enrolled and reporting update status

Description: The target enterprise desktops are enrolled in Intune and are actively reporting Windows Update / driver status.

Why: Windows driver update management only sees and manages devices that check in through Windows Update for Business. A device that is enrolled but not reporting update status cannot be offered or blocked from drivers.

Verify: Intune → **Devices** → **All devices** — the desktops appear and show a recent **Last check-in**. Then **Devices** → **Windows** → **Manage updates** → **Windows Driver updates** — after a policy exists, devices begin appearing under it within 24–48 hours.

Implement if missing: Ensure automatic enrollment is on and devices are Entra-joined or hybrid-joined. If update reporting is absent, confirm the devices are not blocked from `*.windowsupdate.com` / `*.delivery.mp.microsoft.com` and that no legacy WSUS or "Do not connect to any Windows Update locations" policy is overriding cloud updates.

R2. Windows edition and licensing for driver update policies

Description: Windows 10/11 **Pro, Enterprise, or Education** with an Intune-bearing license; driver update management uses Windows Update for Business.

Why: Windows driver update policies are a Windows Update for Business feature; Home edition and devices pointed at WSUS/on-prem-only update sources cannot use them.

Verify: On a sample device run `winver` (edition shown) and confirm in Intune the device is not receiving updates from an on-premises source. In Intune, **Devices** → **Windows** → **Manage updates** → **Windows Driver updates** must be available in the console.

Implement if missing: Upgrade Home to Pro/Enterprise, remove or scope out any WSUS/on-prem update policy for these devices, and confirm the Intune license (per the global requirements) is assigned.

R3. Driver source strategy (Windows Update vs. vendor packages)

Description: A decision on where drivers come from — **Windows Update driver updates** (default for this solution) and, where a device or peripheral needs a driver Windows Update does not publish, a **vendor driver package** delivered as a Win32 app.

Why: Most modern drivers are available through Windows Update and are the safest to stage. A few enterprise peripherals (specialized printers, docks, scanners) publish drivers only through the vendor, which then need packaging.

Verify: For each critical peripheral, check whether its current driver appears under **Windows Driver updates** in Intune (recommended/other lists) or on the vendor's site only.

Implement if missing: Use Windows driver update management for everything Windows Update carries. For the rest, package the vendor driver as a Win32 (`.intunewin`) app and assign it alongside the pilot group — vendor console internals are covered by the vendor's own documentation; the Intune assignment side is what this solution manages.

R4. Printer delivery method (Universal Print or queue package)

Description: A chosen method for mapping printers — **Universal Print** (cloud, no print server) or a **printer deployment package** for a traditional queue.

Why: Peripheral management includes making the right printers available to the right users without manual setup. Universal Print is the cloud-native path; a package is the fallback where a legacy queue must remain.

Verify: If using Universal Print, confirm a Universal Print subscription/licensing is present and at least one printer is registered (Universal Print portal). If using a queue, confirm the print server/queue is reachable from the desktops.

Implement if missing: Register printers in Universal Print and assign a Universal Print connector where needed, **or** prepare a small printer-mapping package for Intune. The Intune assignment (to `PERIDEV-Printers-Pilot`) is documented in the Implementation Guide.

R5. Microsoft Defender device-control prerequisites

Description: Devices are onboarded to Microsoft Defender (Defender Antivirus running, not in passive/disabled state) and **device control is enabled** so removable-storage and USB rules apply.

Why: Defender device control is the engine that enforces allow/audit/deny rules on removable storage and USB. If Defender Antivirus is disabled (e.g., a third-party AV is primary and Defender is off), device-control rules do not enforce.

Verify: On a sample device, run `Get-MpComputerStatus` — `AMRunningMode` should be **Normal** (or EDR block mode as appropriate) and `RealTimeProtectionEnabled` **True**. In Intune, **Endpoint security** → **Antivirus** shows the device as protected.

Implement if missing: Deploy a Defender Antivirus policy so Defender is active, then enable device control via the **Defender** → **Device Control** Settings-Catalog setting (**Device Control Enabled = Enabled**) as part of the device-control policy in the Implementation Guide. If a third-party AV must remain primary, confirm Defender can still run device control before relying on it.

R6. Removable-storage baseline inventory

Description: A read-only picture of how removable/USB storage is used today (which devices, whether any block/allow policy already exists).

Why: You must not block a workflow that a business unit depends on (e.g., an approved encrypted drive) without an exception. Knowing current usage lets you scope the pilot and define the exception group correctly.

Verify: Run `PERIDEV-Discovery.ps1` (read-only) to list existing device-control / removable-storage policies and their assignments; check for existing Administrative-Template "Removable Storage Access" or Defender device-control policies in Intune.

Implement if missing: If no baseline exists, that is fine — the Discovery script establishes it. Record any known approved-drive workflows so they can be added to `SG-PERIDEV-USB-Exception`.

R7. Pilot and production security groups

Description: Entra security groups that target the rollout: a **pilot** group and a **production** group, plus a **USB exception** group for approved removable-storage users/devices.

Why: The whole solution is pilot-first. Groups are how every policy is assigned and expanded. The exception group lets you enforce a strict USB policy fleet-wide while still permitting approved drives for a defined set.

Verify: Entra → **Groups** — confirm these exist with the exact names:

- `SG-PERIDEV-Pilot`
- `SG-PERIDEV-Production`
- `SG-PERIDEV-USB-Exception`

Implement if missing: Create them (Entra → **Groups** → **New group** → **Security**), or run `PERIDEV-Set-ScopeTargeting.ps1` (default `-WhatIf`; re-run with `-Confirm:$true` / `-WhatIf:$false` to apply) to create and populate the pilot targeting. Add a small, representative set of desktops to `SG-PERIDEV-Pilot` first.

R8. A change/rollback window and communications

Description: An agreed maintenance window for the pilot and a simple rollback plan (unassign the policy / set device control back to audit).

Why: Driver and USB changes are user-visible. A window and a stated rollback step keep the pilot low-risk and reversible.

Verify: Confirm with the desktop-operations owner that pilot users know a change is coming and how to report issues.

Implement if missing: Schedule the window; document that rollback = remove the pilot assignment (or switch device control from **Deny** to **Audit**) and force a sync. This is expanded in the Best Practices guide.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Implementation Guide

This guide configures **drivers**, **peripherals/printers**, and **device control (USB/removable)** in Intune, then assigns everything to a **named pilot**, validates it, and expands to production. Follow the steps in order. Every object uses the naming convention from the Solution Overview — **use these names verbatim**.

Sign in to the Microsoft Intune admin center at `https://intune.microsoft.com` as an **Intune Administrator** before you begin.

> ***Deployment model:*** *Assess → Configure → Pilot → Validate → Expand. Do not assign anything to* `SG-PERIDEV-Production` *until the pilot passes the gates in the Testing guide.*

Stage 0 — Assess (read-only)

1. Run `PERIDEV-Discovery.ps1` to capture the current state: existing driver update policies, printer/peripheral configuration profiles, and any removable-storage / Defender device-control policies with their assignments.

2. Note any existing removable-storage policy so you do not conflict with it, and record approved-drive workflows for the exception group.

Stage 1 — Create the groups

If the groups from requirement R7 do not already exist, create them now.

1. Go to `https://entra.microsoft.com` → **Groups** → **All groups** → **New group**.

2. Create each of the following as **Security** groups, **Assigned** membership:

• `SG-PERIDEV-Pilot` — add a small, representative set of desktops (or their users).

• `SG-PERIDEV-Production` — the full target population (leave empty until Stage 6 if you prefer).

• `SG-PERIDEV-USB-Exception` — users/devices allowed to use approved removable storage.

1. (Alternative) Run `PERIDEV-Set-ScopeTargeting.ps1` to create/populate the pilot group. It defaults to `-WhatIf`; re-run with `-WhatIf:$false` to apply.

Stage 2 — Configure Windows driver update management

1. In Intune, go to **Devices** → **Windows** → **Manage updates** → **Windows Driver updates**.

2. Click **Create profile**.

3. **Basics:**

• **Name:** `PERIDEV-DriverUpdates-Pilot`

• **Description:** `Staged driver updates - pilot ring. Manual approval.`

• Click **Next**.

1. **Settings:**

• **Approval method:** select **Manually approve and deploy driver updates**. (Manual approval is the safe pilot behavior — you review each driver before it deploys.)

• Click **Next**.

1. **Scope tags:** leave default (or add your standard scope tag), **Next**.

2. **Assignments:** click **Add groups** and add `SG-PERIDEV-Pilot`. Do **not** add production yet. **Next**.

3. **Review + create** → **Create**.

4. After devices report in (allow 24–48 hours), return to `PERIDEV-DriverUpdates-Pilot`. Under **Driver updates to review**, you will see **Recommended** and **Other** drivers with device counts. For each driver you want to ship to the pilot:

• Select it → **Approve**.

• Set a **Make available** date (immediately, or scheduled to your window).

1. Later, for production, create a second profile named `PERIDEV-DriverUpdates-Prod` the same way. For production you may choose **Automatically approve all** with a deferral (for example, 7 days) once you trust the source, and assign it to `SG-PERIDEV-Production` in Stage 6.

Gate (record in Testing): a pilot device shows the approved driver as installed (Device Manager shows the expected version; `PERIDEV-Validation.ps1` reports driver current).

Stage 3 — Configure peripheral device-installation behavior

This keeps peripheral installation predictable and prevents unexpected device classes from installing.

1. In Intune, go to **Devices** → **Windows** → **Configuration** → **Create** → **New Policy**.

2. **Platform:** Windows 10 and later. **Profile type: Settings catalog**. Click **Create**.

3. **Basics:**

• **Name:** `PERIDEV-Peripherals-DeviceInstall`

• **Description:** `Controls installation of peripheral device classes.`

• **Next**.

1. **Configuration settings:** click **Add settings** and search **Device Installation**. Add the settings you need, for example:

• **Allow installation of devices that match any of these device instance IDs** / **setup classes** — to explicitly allow approved peripherals (printers, docks).

• **Prevent installation of devices not described by other policy settings** — set to **Disabled** for a permissive baseline, or **Enabled** for a locked-down baseline where only approved classes install.

• Configure conservatively for the pilot; a permissive baseline plus explicit allows is the safest start.

1. **Scope tags:** default, **Next**.

2. **Assignments: Add groups** → `SG-PERIDEV-Pilot`. **Next**.

3. **Review + create** → **Create**.

Stage 4 — Deploy printer / peripheral configuration

Choose the method that matches requirement R4.

Option A — Universal Print (cloud, recommended)

1. Register the printer(s) in **Universal Print** (Universal Print portal) and share them.

2. In Intune, go to **Devices** → **Windows** → **Configuration** → **Create** → **New Policy** → **Settings catalog**.

3. **Name:** `PERIDEV-Printers-Pilot`, **Description:** `Maps approved printers to pilot desktops (Universal Print).`

4. **Add settings** → search **Printer Provisioning** and add the **Universal Print** printer-provisioning settings, supplying the shared printer's cloud device ID and name (use the exact values from the Universal Print portal — no angle-bracket placeholders).

5. **Assignments: Add groups** → `SG-PERIDEV-Pilot`. **Review + create** → **Create**.

Option B — Traditional queue (fallback)

1. Prepare a small printer-mapping package (script or Win32 app) that connects the desktop to the existing print queue by its UNC path (for example `\\print-server-01\FINANCE-MFP`).

2. In Intune, go to **Apps** → **Windows** → **Add** → **Windows app (Win32)**, upload the `.intunewin`, and set install/uninstall commands.

3. **Name:** `PERIDEV-Printers-Pilot`.

4. **Assignments: Required** → `SG-PERIDEV-Pilot`. Save.

Gate (record in Testing): a pilot desktop shows the printer mapped and can print a test page (`PERIDEV-Validation.ps1` reports printer mapped).

Stage 5 — Configure device control (USB / removable storage)

Pick the surface that matches your requirement. For a pilot, **start in Audit** so you see impact before enforcing.

Option A — Defender device control (rich rules, exceptions) — recommended

1. In Intune, go to **Endpoint security** → **Attack surface reduction** (or **Devices** → **Windows** → **Configuration** → **Create** → **New Policy** → **Settings catalog**).

2. **Platform:** Windows 10 and later. **Profile type: Settings catalog** (Defender Device Control). Click **Create**.

3. **Basics:**

• **Name:** `PERIDEV-DeviceControl-USB-Pilot`

• **Description:** `Removable-storage control - pilot. Audit first, then Deny write.`

• **Next**.

1. **Configuration settings: Add settings** → search **Device Control**:

• **Device Control Enabled** → **Enabled** (this turns the engine on — see requirement R5).

• Configure a rule for the **Removable storage** device family:

• For the initial pilot, set the removable-storage access to **Audit** (write/read/execute audited) so you observe usage without blocking.

• When ready to enforce, change to **Deny write** (block writing to removable storage, still allow read) or **Deny all** per your requirement.

• If you need an exception (approved encrypted drives for `SG-PERIDEV-USB-Exception`), define an **allow** rule/group for those device IDs so the exception group retains access while everyone else is denied.

1. **Scope tags:** default, **Next**.

2. **Assignments: Add groups** → `SG-PERIDEV-Pilot`. **Next**.

3. **Review + create** → **Create**.

Option B — Administrative Templates / Settings Catalog "Removable Storage Access" (simple blanket block)

Use this when you only need a blanket control and no per-device exceptions.

1. **Devices** → **Windows** → **Configuration** → **Create** → **New Policy** → **Settings catalog.**

2. **Name:** `PERIDEV-DeviceControl-USB-Pilot`.

3. **Add settings** → search **Removable Storage Access** and set, for example, **Removable Disks: Deny write access = Enabled** (and Deny read/execute as required).

4. **Assignments: Add groups** → `SG-PERIDEV-Pilot`. **Review + create** → **Create**.

Gate (record in Testing): on a pilot device, inserting a non-approved USB drive is audited (Audit phase) or blocked (Enforce phase); an approved drive for `SG-PERIDEV-USB-Exception` still works. `PERIDEV-Validation.ps1` reports USB policy enforced.

Stage 6 — Validate, then expand to production

1. **Validate the pilot** against all three gates using the Testing guide and `PERIDEV-Validation.ps1`:

• Driver current (Stage 2 gate)

• Printer mapped (Stage 4 gate)

• USB policy enforced (Stage 5 gate)

1. Run `PERIDEV-Report.ps1` to confirm coverage across the pilot group.

2. If all gates **pass**, switch the device-control policy from **Audit** to **Deny** on the pilot and re-validate the USB gate.

3. **Expand:**

• Create `PERIDEV-DriverUpdates-Prod` (Stage 2, step 9) and assign to `SG-PERIDEV-Production`.

• Duplicate the peripheral, printer, and device-control policies as `...-Prod` (or add `SG-PERIDEV-Production` to the assignment) and assign to `SG-PERIDEV-Production`.

• Keep `SG-PERIDEV-Pilot` ahead of production for all future changes.

1. Run `PERIDEV-Report.ps1` again against production to confirm coverage after expansion.

> ***Rollback at any point:*** *remove the pilot/production assignment from the affected policy (or switch device control from* ***Deny*** *back to* ***Audit****) and force a device sync (****Devices*** *→* ***sync****, or* `PERIDEV-Remediation.ps1`*). Nothing in this solution requires an irreversible change.*

Testing — Pass/Fail Gates

Testing confirms the solution works on the **pilot** before you expand, and confirms it still works on **production** after expansion. Each gate has an explicit **pass/fail** condition. Do not expand until all three gates **pass** on the pilot.

Run `PERIDEV-Validation.ps1` for the automated per-device view, and confirm each gate manually on at least one pilot device.

Gate 1 — Driver current

What it proves: the approved driver update from `PERIDEV-DriverUpdates-Pilot` actually installed on pilot devices.

How to test:

1. On a pilot desktop, open **Device Manager** → find the device you approved a driver for → **Properties** → **Driver** tab.
2. Confirm the **Driver Version / Date** matches the version you approved in Intune.
3. In Intune, open `PERIDEV-DriverUpdates-Pilot` → the approved driver shows devices as **up to date / installed**.
4. Run `PERIDEV-Validation.ps1` and read the **Driver** column for the device.

Pass: the installed driver version equals the approved version on the pilot device(s), and Intune reports the driver installed. **Fail:** the device still runs the old version after the make-available date and a sync, or the driver never appears — see Troubleshooting.

Gate 2 — Printer mapped

What it proves: the printer delivered by `PERIDEV-Printers-Pilot` is available and usable on pilot devices.

How to test:

1. On a pilot desktop, open **Settings** → **Bluetooth & devices** → **Printers & scanners**.
2. Confirm the approved printer appears (Universal Print printers show as cloud printers; queue-mapped printers show by their queue name).
3. Print a **Windows test page** to that printer and confirm it prints.
4. Run `PERIDEV-Validation.ps1` and read the **Printer** column.

Pass: the printer is present and a test page prints successfully on the pilot device(s). **Fail:** the printer is missing, shows an error, or the test page does not print — see Troubleshooting.

Gate 3 — USB / removable-storage policy enforced

What it proves: `PERIDEV-DeviceControl-USB-Pilot` behaves as configured — auditing in the Audit phase, blocking in the Enforce phase — and the exception group still works.

How to test (Audit phase):

1. Insert a standard (non-approved) USB flash drive into a pilot desktop.
2. Confirm access is still allowed **and** an audit event is recorded (Defender device-control audit / event log).

How to test (Enforce phase):

1. Insert a standard (non-approved) USB flash drive into a pilot desktop.
2. Attempt to **write** a file to it. Confirm the write is **blocked** (and read allowed, or blocked, per your rule).

3. On a device in `SG-PERIDEV-USB-Exception`, insert an **approved** drive and confirm it works as intended.

4. Run `PERIDEV-Validation.ps1` and read the **DeviceControl** column.

Pass:

- Audit phase: non-approved drive is allowed and the access is audited.
- Enforce phase: non-approved drive is blocked per the rule, and an approved drive for `SG-PERIDEV-USB-Exception` still works.

Fail: a non-approved drive is not blocked in the Enforce phase, an approved drive is wrongly blocked, or no audit/block events appear — see Troubleshooting.

Coverage check (before expanding)

Run `PERIDEV-Report.ps1` against `SG-PERIDEV-Pilot`. Confirm:

- Every pilot device is covered by the driver, printer, and device-control policies (no "not applied" gaps).
- No device reports an **error** state for any of the three policies.

Pass: 100% of pilot devices covered by all three policy areas with no error states. **Fail:** any pilot device is missing a policy or in error — resolve before expanding.

Post-expansion check (production)

After Stage 6 expansion, re-run `PERIDEV-Report.ps1` against `SG-PERIDEV-Production` and re-confirm Gates 1–3 on a sample of production devices. Coverage should trend toward 100% as devices sync.

Troubleshooting

Symptom → likely cause → fix, organized by the three areas. Use `PERIDEV-Validation.ps1` and `PERIDEV-Report.ps1` to confirm state, and remember every change is reversible (unassign the policy or switch device control back to **Audit**, then sync).

Drivers

A driver never appears under `PERIDEV-DriverUpdates-Pilot` to approve

- **Cause:** devices haven't reported update status yet, or the device isn't using Windows Update for Business.
- **Fix:** allow 24–48 hours after first assignment. Confirm the device is not pointed at WSUS/on-prem updates (requirement R2), reaches `*.windowsupdate.com`, and is Pro/Enterprise/Education (requirement R2). Force a sync (`PERIDEV-Remediation.ps1`).

Approved driver isn't installing on a pilot device

• **Cause:** make-available date is in the future, the device hasn't synced, or the device already has an equal/newer driver.

• **Fix:** check the **Make available** date on the approval, force a sync, and confirm in Device Manager the device isn't already at/above that version. Verify via Gate 1 in Testing.

A driver caused a problem after approval

• **Cause:** a bad driver reached the pilot — exactly what the pilot is for.

• **Fix:** on the device, Device Manager → **Roll Back Driver**. In Intune, withdraw/don't-expand the approval. Do **not** approve it to `PERIDEV-DriverUpdates-Prod`. Investigate before re-approving.

Devices getting drivers you didn't approve

• **Cause:** production policy set to **Automatically approve all** with too small a deferral, or another update policy is competing.

• **Fix:** increase the deferral on `PERIDEV-DriverUpdates-Prod`, or switch it to manual approval. Run `PERIDEV-Discovery.ps1` to find competing update policies.

Peripherals & printers

Printer from `PERIDEV-Printers-Pilot` doesn't appear on the device

• **Cause (Universal Print):** printer not shared, wrong cloud device ID in the profile, or user lacks access.

• **Cause (queue):** UNC path unreachable, or the mapping package failed.

• **Fix:** re-check the Universal Print printer's shared status and that the exact cloud device ID/name is in the profile (no placeholder text). For a queue, confirm the print server path (for example `\\print-server-01\FINANCE-MFP`) resolves from the device. Force a sync and re-test Gate 2.

Printer appears but test page fails

• **Cause:** driver missing, or the printer offline.

• **Fix:** confirm the print device is online; if a vendor driver is required, deliver it as a Win32 app alongside the pilot (requirement R3).

An unexpected peripheral installed (or a needed one is blocked)

• **Cause:** `PERIDEV-Peripherals-DeviceInstall` is too permissive or too strict.

• **Fix:** add the approved setup class/instance ID to the allow list; if using the locked-down "prevent installation of devices not described by other policy settings," ensure every approved peripheral is explicitly allowed. Re-sync.

Device control (USB / removable)

USB drive is not blocked in the Enforce phase

• **Cause:** policy still in **Audit**, Defender inactive, device not synced, or device outside `SG-PERIDEV-Pilot`.

• **Fix:** confirm `PERIDEV-DeviceControl-USB-Pilot` is set to **Deny** (not Audit) and that **Device Control Enabled = Enabled**. Run `Get-MpComputerStatus` — Defender must be active (requirement R5). Confirm the device is in the pilot group and synced. Re-test Gate 3.

An approved drive is wrongly blocked

• **Cause:** the device/user isn't in `SG-PERIDEV-USB-Exception`, or the allow rule doesn't match the drive's device ID.

• **Fix:** add the user/device to `SG-PERIDEV-USB-Exception` and confirm the allow rule's device IDs match the approved drive. Re-sync and re-test.

No audit or block events appear at all

• **Cause:** device control not enabled, or Defender Antivirus disabled/passive.

• **Fix:** set **Device Control Enabled = Enabled**; ensure a Defender Antivirus policy has Defender in **Normal** running mode (requirement R5). If a third-party AV is primary and Defender is off, device control will not enforce.

Policy shows conflict/error in the report

• **Cause:** two policies set the same removable-storage setting differently (e.g., an Administrative-Template block and a Defender device-control rule).

• **Fix:** run `PERIDEV-Discovery.ps1` to find overlapping removable-storage policies; keep one authoritative surface (Defender device control **or** the Removable Storage Access template), not both. Remove the duplicate and re-sync.

Coverage, assignment, and scripts

A device is "not covered" in `PERIDEV-Report.ps1`

• **Cause:** device not in the assigned group, or hasn't synced.

• **Fix:** confirm membership of `SG-PERIDEV-Pilot`/`SG-PERIDEV-Production`; run `PERIDEV-Remediation.ps1` (default `-WhatIf`) to review, then `-WhatIf:$false` to assign and sync.

A script fails to connect to Graph

• **Cause:** SDK not installed, or missing consent for the required scopes.

• **Fix:** `Install-Module Microsoft.Graph -Scope CurrentUser`; sign in with an **Intune Administrator** and consent to the scopes listed in each script's HOW TO RUN section.

Validation shows FAIL but the device looks fine manually

• **Cause:** reporting lag — device configuration state hasn't refreshed in Intune yet.

• **Fix:** force a sync, wait for the device to report, and re-run `PERIDEV-Validation.ps1`. Confirm the gate manually per the Testing guide.

Rollback needed

- **Cause:** any pilot change caused unacceptable impact.
- **Fix:** remove the group assignment from the affected policy (or switch device control from **Deny** to **Audit**), then force a sync (`PERIDEV-Remediation.ps1` or **Devices** → **sync**). Nothing in this solution requires an irreversible change.

Best Practices

These practices keep peripheral, driver, and device-control management safe, reversible, and low-noise as you run it over time.

Staged driver rollout

- **Always pilot first.** Approve drivers only to `PERIDEV-DriverUpdates-Pilot` (manual approval), let them **soak** for a defined period (for example, 3–7 days) on `SG-PERIDEV-Pilot`, then approve to production via `PERIDEV-DriverUpdates-Prod`.
- **Manual approval for pilot, deferred-automatic for production.** Use **Manually approve and deploy** on the pilot so you review every driver. For production, once a source is trusted, **Automatically approve all** with a **deferral** (e.g., 7 days) reduces toil while keeping a safety gap.
- **One change at a time in the pilot.** Don't approve a batch of drivers and change USB policy in the same window — if something breaks, you want to know which change caused it.
- **Prefer Windows Update drivers over vendor packages.** They are staged, revertible, and reported in Intune. Package a vendor driver only when Windows Update does not carry it.
- **Watch device counts.** In the driver-review list, a driver needed by only a few devices is lower risk to approve first; a driver touching the whole fleet deserves a longer pilot soak.
- **Know the rollback.** To roll back a driver, unassign/withdraw the approval and, if needed, roll back the driver on the device (Device Manager → **Roll Back Driver**). To roll back a policy, remove the group assignment and force sync (`PERIDEV-Remediation.ps1`).

Device control (USB / removable) exceptions

- **Audit before you enforce.** Every device-control policy should ship to the pilot in **Audit** first. Read the audit events, confirm you understand real usage, then switch to **Deny**.
- **Enforce broadly, allow narrowly.** The clean pattern is: **Deny** removable storage for everyone, then **allow** an explicit exception for `SG-PERIDEV-USB-Exception` (approved, ideally encrypted, drives). Keep the exception group small and reviewed.
- **Prefer "Deny write" over "Deny all" when the goal is data-loss prevention.** Blocking writes stops exfiltration while still letting users read from vendor/media USB devices, which cuts help-desk noise.
- **Document every exception.** For each member of `SG-PERIDEV-USB-Exception`, record who approved it, why, and a review date. Exceptions are the part of device control that drifts.
- **Keep Defender healthy.** Device control only enforces while Defender Antivirus is active (requirement R5). Include Defender health in your coverage checks so a disabled AV doesn't silently drop

enforcement.

Peripherals and printers

- **Explicit-allow beats implicit-block for peripheral installation.** Maintain a known list of approved peripheral setup classes/IDs in `PERIDEV-Peripherals-DeviceInstall`; a locked-down "block everything not listed" baseline should only follow a clean audit.
- **Prefer Universal Print** for new printer deployments — no print server to maintain, and it maps cleanly through `PERIDEV-Printers-Pilot`. Keep queue packages only where a legacy queue must remain.
- **Name printers consistently** so users and the validation script can recognize the expected mapping.

Operating the solution

- **Keep the naming convention verbatim.** `SG-PERIDEV-*`, `PERIDEV-DriverUpdates-*`, `PERIDEV-Printers-*`, `PERIDEV-DeviceControl-USB-*`, `PERIDEV-Peripherals-DeviceInstall`. The scripts and every guide assume these exact names; renaming breaks reporting and remediation.
- **Keep the pilot ahead of production, permanently.** Every future driver, printer, or USB change goes to the pilot first — not just the initial rollout.
- **Run the read-only scripts on a schedule.** `PERIDEV-Discovery.ps1` and `PERIDEV-Report.ps1` are safe to run regularly to catch drift (a device that fell out of coverage, a disabled Defender, an unexpected new removable-storage policy).
- **Default to `-WhatIf` for anything that changes state.** `PERIDEV-Remediation.ps1` and `PERIDEV-Set-ScopeTargeting.ps1` default to `-WhatIf`; review the planned actions before re-running to apply.
- **Communicate user-visible changes.** Drivers and USB blocking are noticed. A short heads-up to the pilot and a known way to report issues keeps confidence high.

Deliverables Summary

This solution is a complete, self-guided package for managing **drivers, peripherals/printers, and removable-device control** on enterprise desktops with Microsoft Intune — deployed pilot-first and rollback-ready.

Documents

Document	Purpose
Solution Overview	How Intune manages drivers (Windows driver update management), peripherals/printers, and device control (removable-storage access); the standard deployment model and object naming convention.

Document	Purpose
Global Requirements — Intune Solutions	Category-level prerequisites (tenant, licensing, roles, enrollment, Windows baseline, network) shared by all Intune solutions.
Requirements	Solution-specific prerequisites (R1–R8), each with Description / Why / Verify / Implement-if-missing.
Implementation Guide	Exact, ordered Intune steps: assess, create groups, configure driver update management, peripheral device-installation, printer delivery, and device-control (USB/removable) policy; assign to the pilot; validate; expand.
Testing	Pass/fail gates — driver current, printer mapped, USB policy enforced — plus coverage and post-expansion checks.
Troubleshooting	Symptom → cause → fix for the common failure modes across all three areas.
Best Practices	Staged driver rollout, device-control exceptions, peripheral/printer hygiene, and operating the solution over time.

PowerShell scripts

All scripts use the Microsoft Graph PowerShell SDK. Discovery, Validation, and Report are **read-only**. Remediation and Set-ScopeTargeting default to `-WhatIf` and must be re-run to apply.

Script	Function	Mode
`PERIDEV-Discovery.ps1`	Inventory driver update policies, printer/peripheral configuration profiles, and removable-storage / device-control policies with assignments.	Read-only
`PERIDEV-Validation.ps1`	Per-device applied state for the three gates (driver current, printer mapped, USB policy enforced).	Read-only
`PERIDEV-Report.ps1`	Coverage report across a target group — which devices are covered by each policy area and any error states.	Read-only
`PERIDEV-Remediation.ps1`	Safely fix missing/blocked coverage (re-assign policy, trigger sync) for devices out of coverage.	`-WhatIf` default

Script	Function	Mode
`PERIDEV-Set-ScopeTargeting.ps1`	Create/populate the pilot (and related) security groups for targeting.	`-WhatIf` default

Object naming convention (reused everywhere)

Object	Name
Pilot security group	`SG-PERIDEV-Pilot`
Production security group	`SG-PERIDEV-Production`
USB exception group	`SG-PERIDEV-USB-Exception`
Driver update policy — pilot	`PERIDEV-DriverUpdates-Pilot`
Driver update policy — production	`PERIDEV-DriverUpdates-Prod`
Printer/peripheral profile — pilot	`PERIDEV-Printers-Pilot`
Peripheral device-installation profile	`PERIDEV-Peripherals-DeviceInstall`
Device-control (USB) policy — pilot	`PERIDEV-DeviceControl-USB-Pilot`
Device-control (USB) policy — production	`PERIDEV-DeviceControl-USB-Prod`

Outcome

Drivers current, peripherals working, and removable devices controlled across the fleet — managed from Intune, piloted before production, validated against explicit pass/fail gates, and reversible at every step, with scripts you keep and reuse.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PERIDEV-Discovery.ps1

Purpose: Read-only discovery of the current state for peripheral, driver, and device-control management. Lists Windows driver update policies, printer/peripheral configuration profiles, and removable-storage / Defender device-control policies, with their group assignments, so you can assess before you configure. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PERIDEV-Discovery.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\PERIDEV-Discovery.ps1

PERIDEV-Remediation.ps1

Purpose: Safe remediation for devices out of coverage: ensures the target group is assigned to the PERIDEV driver, printer, and device-control policies, and triggers a device sync so fixes apply. Defaults to preview mode (-WhatIf) and changes nothing until you re-run to confirm. Only re-assigns known PERIDEV policies and syncs devices - it never blocks, deletes, or reconfigures policy rules. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\PERIDEV-Remediation.ps1

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\PERIDEV-Remediation.ps1

PERIDEV-Report.ps1

Purpose: Read-only coverage report. For a target group (pilot or production), summarizes how many devices are covered by the driver, printer/peripheral, and device-control policies and flags any error states, so you can confirm coverage before expanding and catch drift over time. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PERIDEV-Report.ps1 powershell -ExecutionPolicy Bypass -File .\PERIDEV-Report.ps1 -GroupName SG-PERIDEV-Production -OutCsv coverage.csv

How to run: powershell -ExecutionPolicy Bypass -File .\PERIDEV-Report.ps1 powershell -ExecutionPolicy Bypass -File .\PERIDEV-Report.ps1 -GroupName SG-PERIDEV-Production -OutCsv coverage.csv

PERIDEV-Set-ScopeTargeting.ps1

Purpose: Sets up the security groups this solution targets: the pilot group, the production group, and the USB exception group, and optionally adds seed devices to the pilot. Defaults to preview mode (-WhatIf) and changes nothing until you re-run to confirm. Uses the exact PERIDEV names so all guides and scripts line up. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\PERIDEV-Set-ScopeTargeting.ps1

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\PERIDEV-Set-ScopeTargeting.ps1

PERIDEV-Validation.ps1

Purpose: Read-only per-device validation of the peripheral, driver, and device-control gates: driver current, printer mapped, and USB/device-control policy enforced. For each managed device (optionally scoped to a group), it reports the applied state of the PERIDEV policies so you can confirm the pilot before

you expand. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PERIDEV-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\PERIDEV-Validation.ps1 -GroupName SG-PERIDEV-Pilot

How to run: powershell -ExecutionPolicy Bypass -File .\PERIDEV-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\PERIDEV-Validation.ps1 -GroupName SG-PERIDEV-Pilot